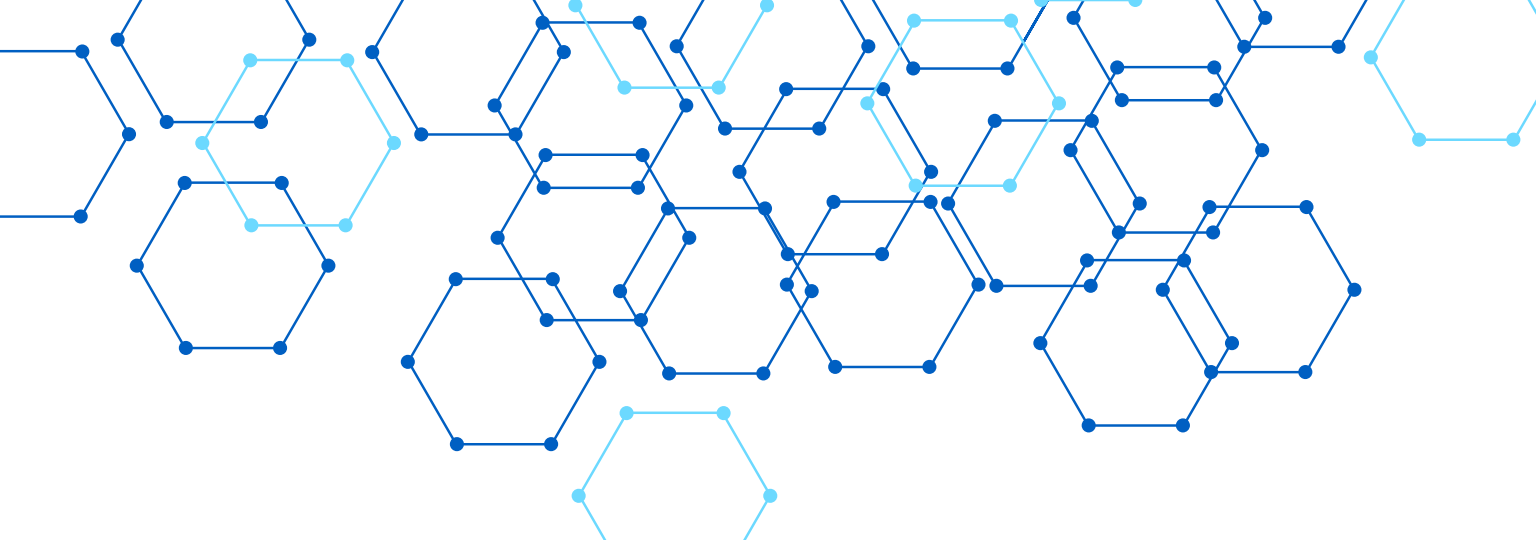
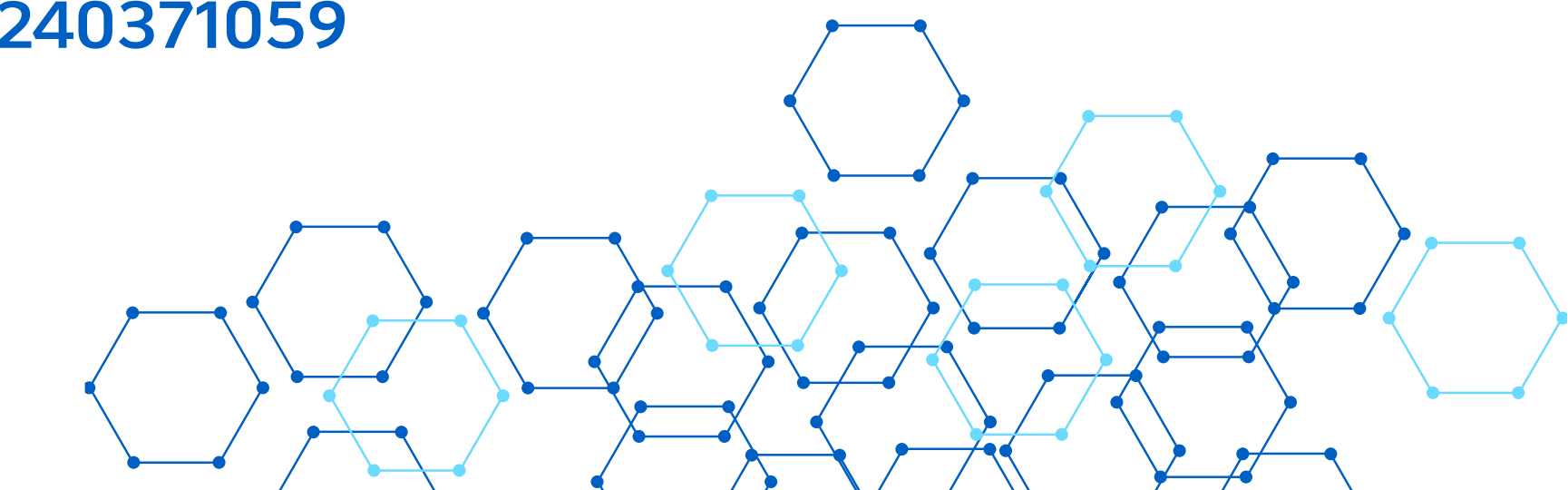




IMPLEMENTACIÓN Y COMPARACIÓN DE MÉTODOS NUMÉRICOS PARA BÚSQUEDA DE RAÍCES DE POLINOMIOS

David Santiago Vargas Parra - 01240371048
Moises David Baquero Daza - 01240371051
Jose Daniel Nova Muñoz - 01240371059



OBJETIVO GENERAL



Desarrollar una aplicación de escritorio en Python que implemente y compare los métodos numéricos abiertos Newton-Raphson y Müller para la búsqueda de raíces de polinomios, visualizando el proceso iterativo y el comportamiento de convergencia de cada método.

OBJETIVOS ESPECIFICOS

1. Investigar y documentar los fundamentos matemáticos de los métodos de Newton-Raphson y Müller, incluyendo su derivación, criterios de convergencia y casos de fallo.
2. Diseñar una arquitectura modular orientada a objetos que separe la representación del polinomio, la lógica de los métodos numéricos y la interfaz gráfica de usuario.
3. Implementar los algoritmos de Newton-Raphson y Müller con soporte para números complejos, control de tolerancia y límite de iteraciones.
4. Construir una interfaz gráfica con Tkinter que permita ingresar parámetros, ejecutar ambos métodos y visualizar los resultados en tablas comparativas y diagrama de polos.
5. Evaluar y comparar el rendimiento de ambos métodos en términos de número de iteraciones, error final y tiempo de cómputo para distintos polinomios de prueba.

PROBLEMA IDENTIFICADO Y MOTIVACIÓN



Existe una brecha significativa entre la teoría y la práctica en la enseñanza de métodos numéricos. Los estudiantes enfrentan dificultades para observar la evolución del error en cada iteración, comprender el comportamiento de raíces complejas y evaluar la eficiencia real de los algoritmos en tiempo de ejecución.



Las herramientas actuales como MATLAB y Wolfram Alpha, aunque potentes, presentan limitaciones para el aprendizaje pedagógico: costos de licencia, interfaces complejas y falta de visualización paso a paso. Surge la necesidad de una herramienta educativa específica, gratuita y visual que facilite la comprensión profunda de estos métodos.

MÉTODO NEWTON-RAPHSON

$x_{n+1} = x_n - \frac{P(x_n)}{P'(x_n)}$

Fórmula iterativa: $x_{n+1} = x_n - P(x_n)/P'(x_n)$. Convergencia cuadrática que requiere el cálculo de la derivada del polinomio en cada iteración.



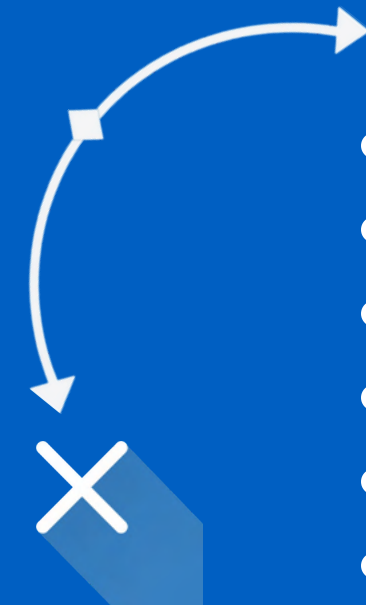
Ventajas: Convergencia muy rápida cerca de la raíz, requiere pocas iteraciones para alcanzar precisión alta. Criterio de parada basado en error relativo porcentual.



Limitaciones: Falla cuando la derivada es cero o cercana a cero. No puede encontrar raíces complejas partiendo de puntos iniciales reales.

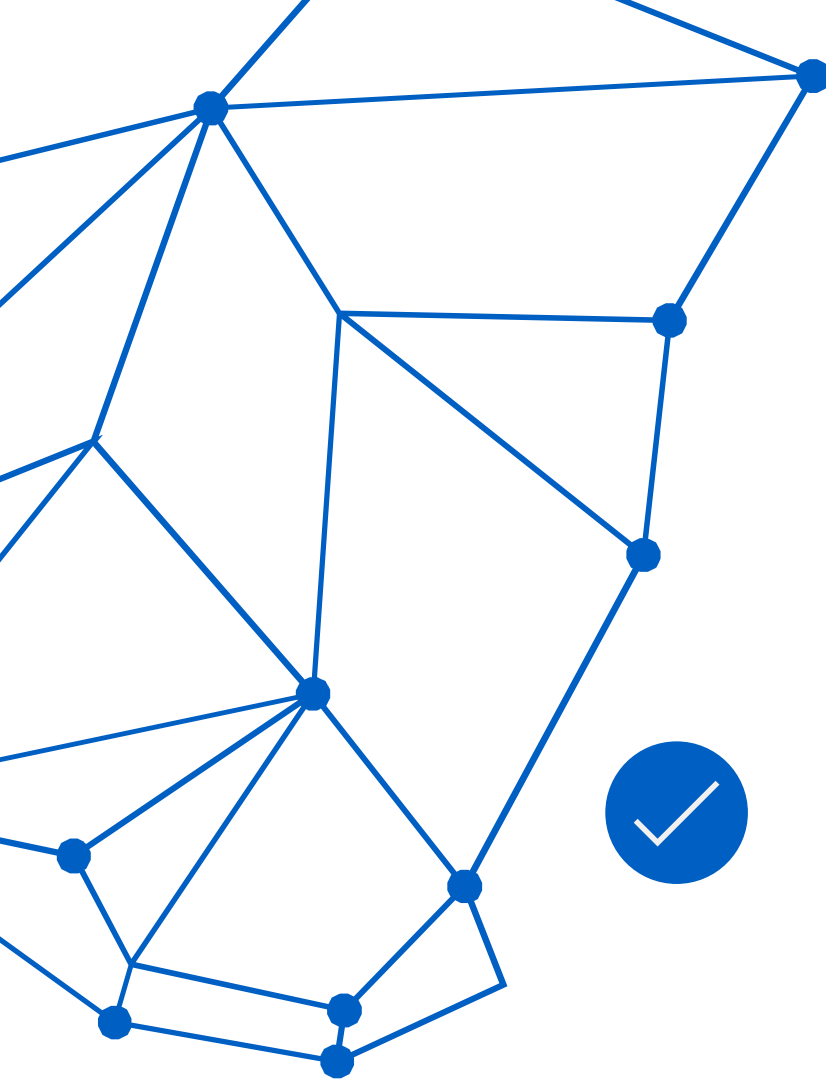
MÉTODO DE MÜLLER

El método de Müller permite analizar el comportamiento dinámico del sistema y detectar posibles inestabilidades en el tiempo de respuesta.

- 
- Usa tres puntos iniciales (x_0, x_1, x_2) para construir una parábola.
 - No requiere derivadas.
 - Convergencia superlineal ≈ 1.84 .
 - Funciona bien con funciones irregulares o con ruido.
 - Puede encontrar raíces complejas.
 - Detecta comportamientos oscilatorios e inestables del sistema.



Müller complementa a Newton al analizar estabilidad y oscilaciones del sistema.

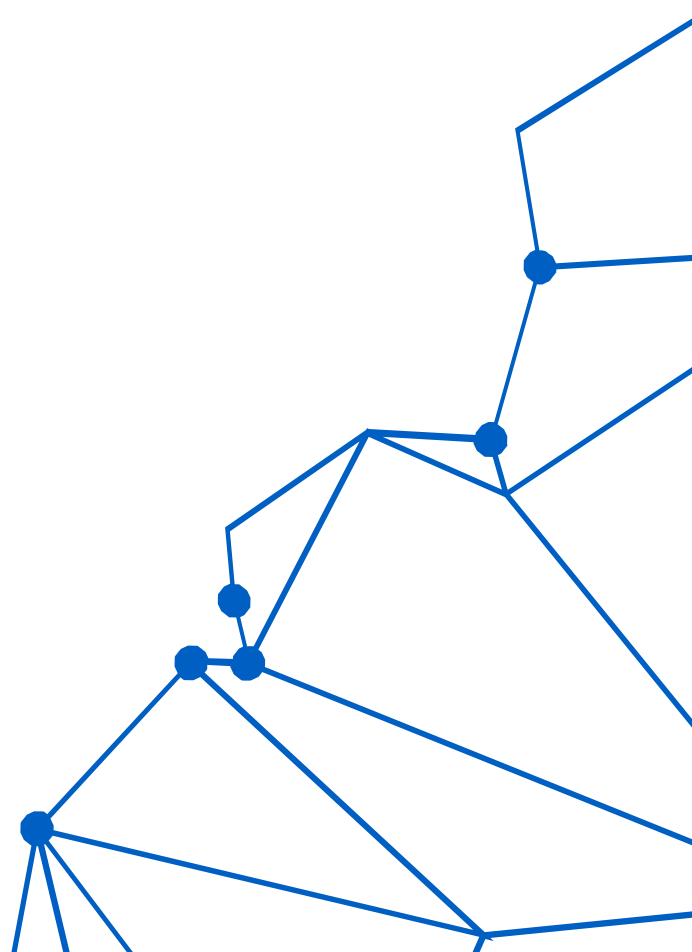


COMPARACIÓN ENTRE NEWTON-RAPHSON Y MÜLLER



Ambos métodos numéricos permiten analizar el rendimiento del sistema desde perspectivas diferentes y complementarias.

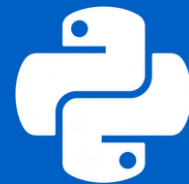
Newton-Raphson	Método de Müller
Convergencia cuadrática	Convergencia superlineal (1.84)
Requiere derivadas	No requiere derivadas
Solo raíces reales	Puede encontrar raíces complejas
Más rápido	Más robusto
Ideal para funciones suaves	Ideal para funciones irregulares



ARQUITECTURA DEL SISTEMA Y STACK TECNOLÓGICO



Arquitectura modular en 4 capas: Capa Polinomio (representación y operaciones), Capa Métodos Numéricos (Newton-Raphson y Müller), Capa Interfaz Gráfica (Tkinter), y Capa Main (orquestración del sistema).



Python 3.11 como lenguaje base con soporte nativo para números complejos. NumPy para evaluación eficiente de polinomios. SymPy para derivación simbólica automática requerida por Newton-Raphson.



Matplotlib para visualización del diagrama de polos y ceros en el plano complejo. Tkinter como framework para la interfaz gráfica de usuario, ofreciendo una experiencia interactiva y accesible.

FUNCIONALIDADES Y METODOLOGÍA



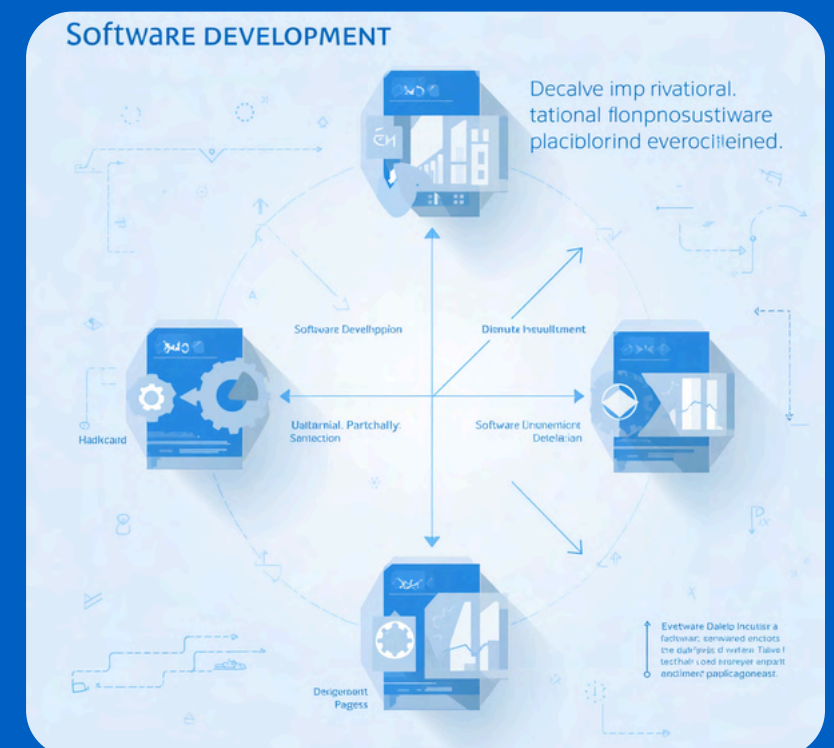
Soporte para polinomios de grado ≥ 1 con entrada por coeficientes. Newton-Raphson implementado con derivada simbólica (SymPy) y punto inicial complejo configurable.

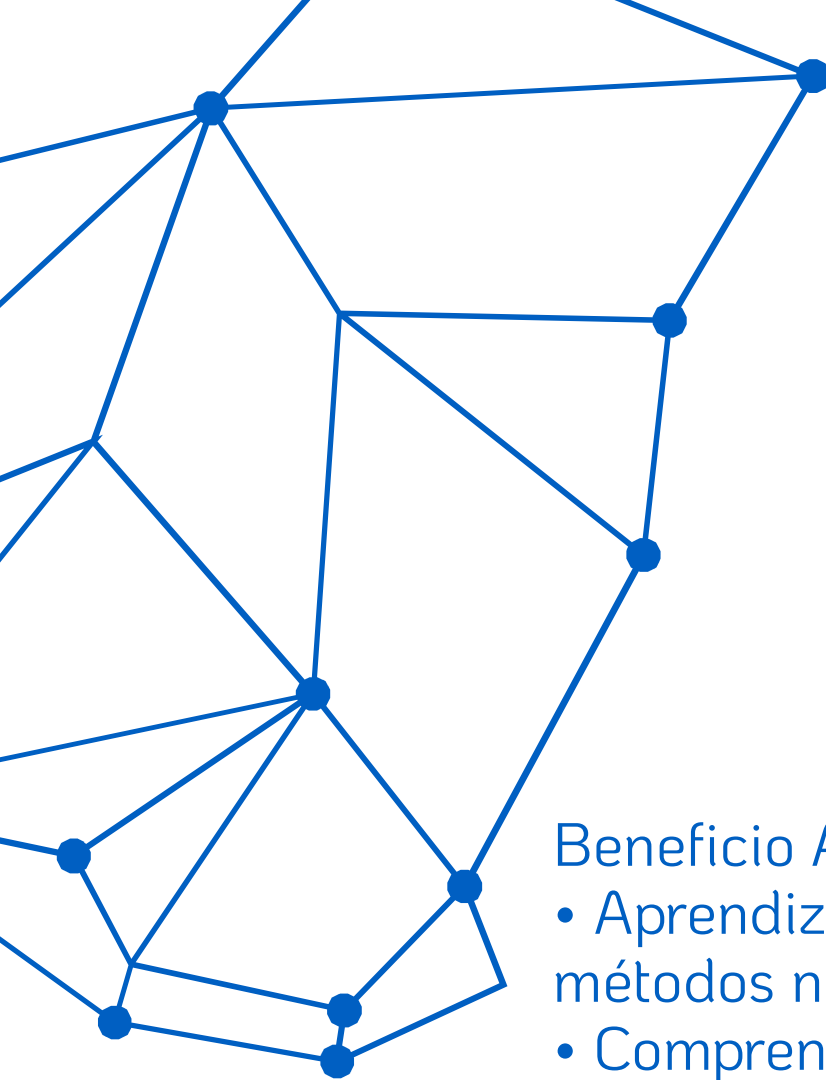


Método de Müller con aritmética compleja nativa y tres puntos iniciales configurables. Visualización de iteraciones, errores relativos porcentuales y diagrama de polos en plano complejo.



Metodología de prototipado evolutivo en 4 fases: fundamentación teórica, diseño modular, prototipo en consola, e interfaz gráfica con pruebas de validación.





BENEFICIOS, IMPACTO Y RECOMENDACIONES FUTURAS

Beneficio Académico

- Aprendizaje visual e interactivo de métodos numéricos
 - Comprensión profunda de convergencia iterativa
 - Visualización de raíces complejas en diagrama de polos
 - Alternativa gratuita a herramientas comerciales
 - Puente entre teoría matemática y práctica computacional
- 



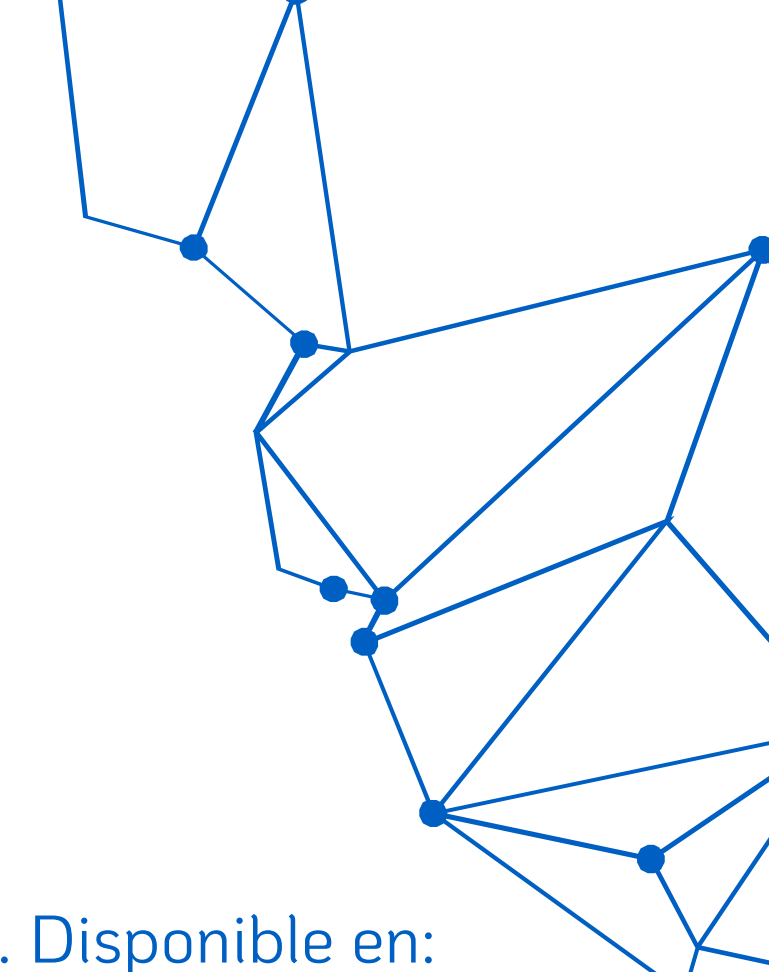
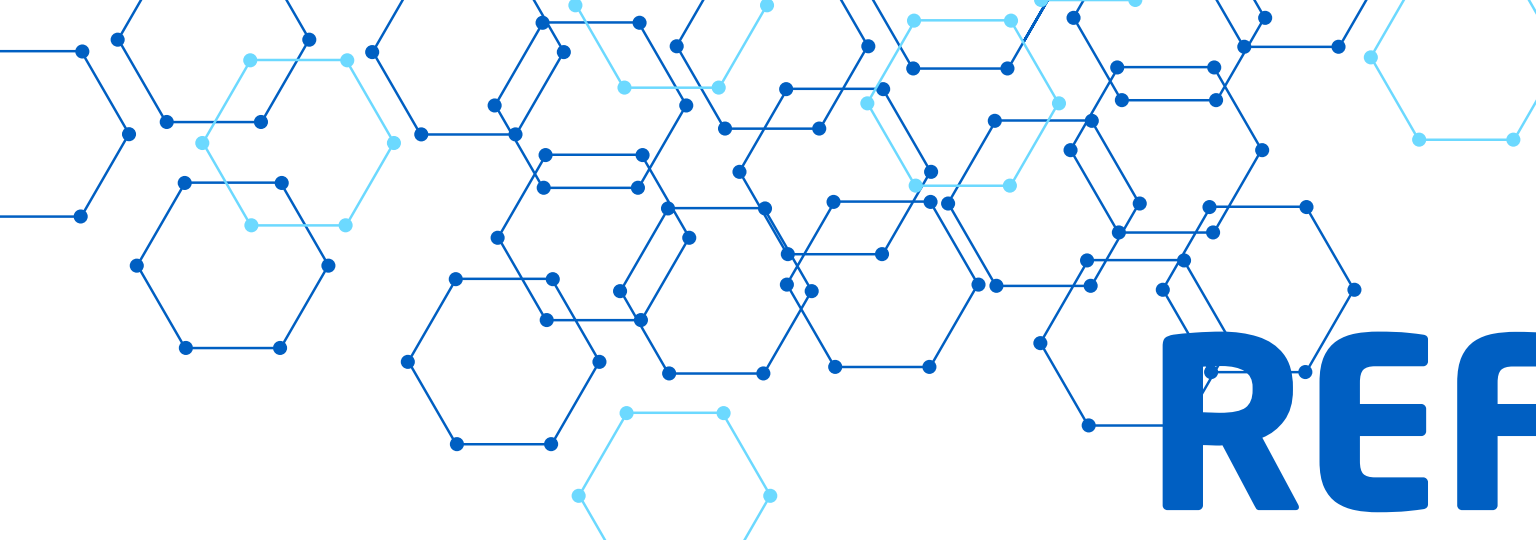
Recomendaciones Futuras

- Implementar deflación de polinomios para raíces múltiples
- Agregar método Bairstow para factores cuadráticos
- Desarrollar gráficas iterativas animadas
- Exportación de resultados a CSV/PDF
- Ampliar banco de casos de prueba

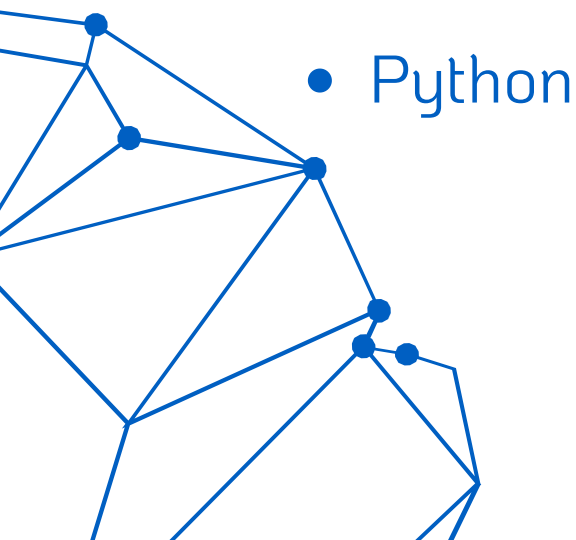
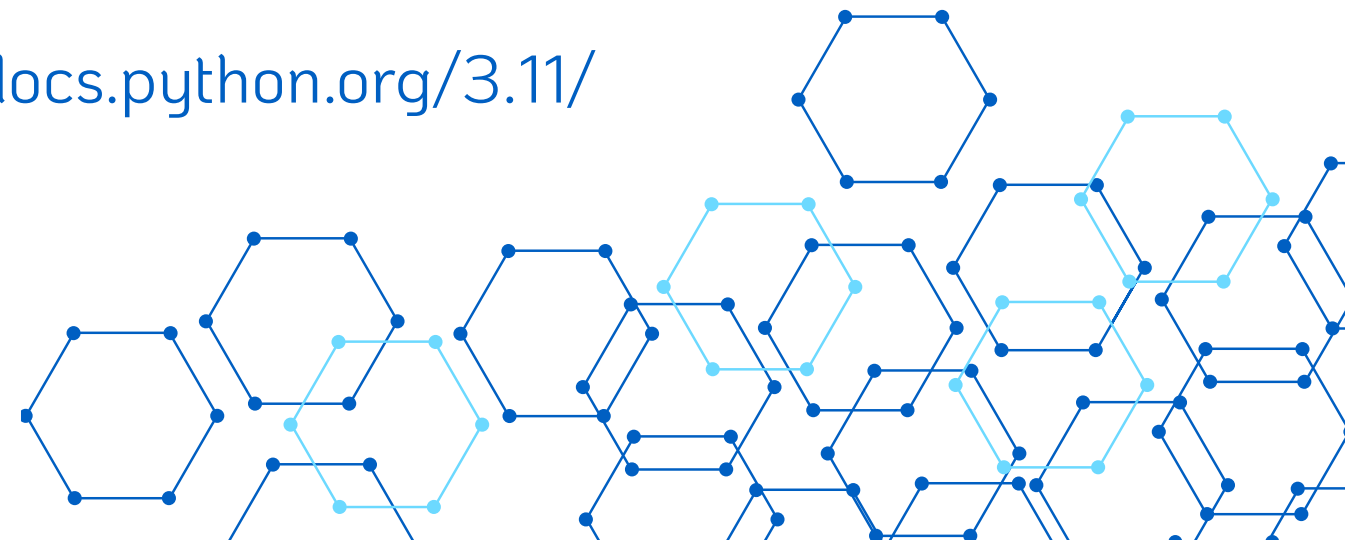
Impacto Técnico

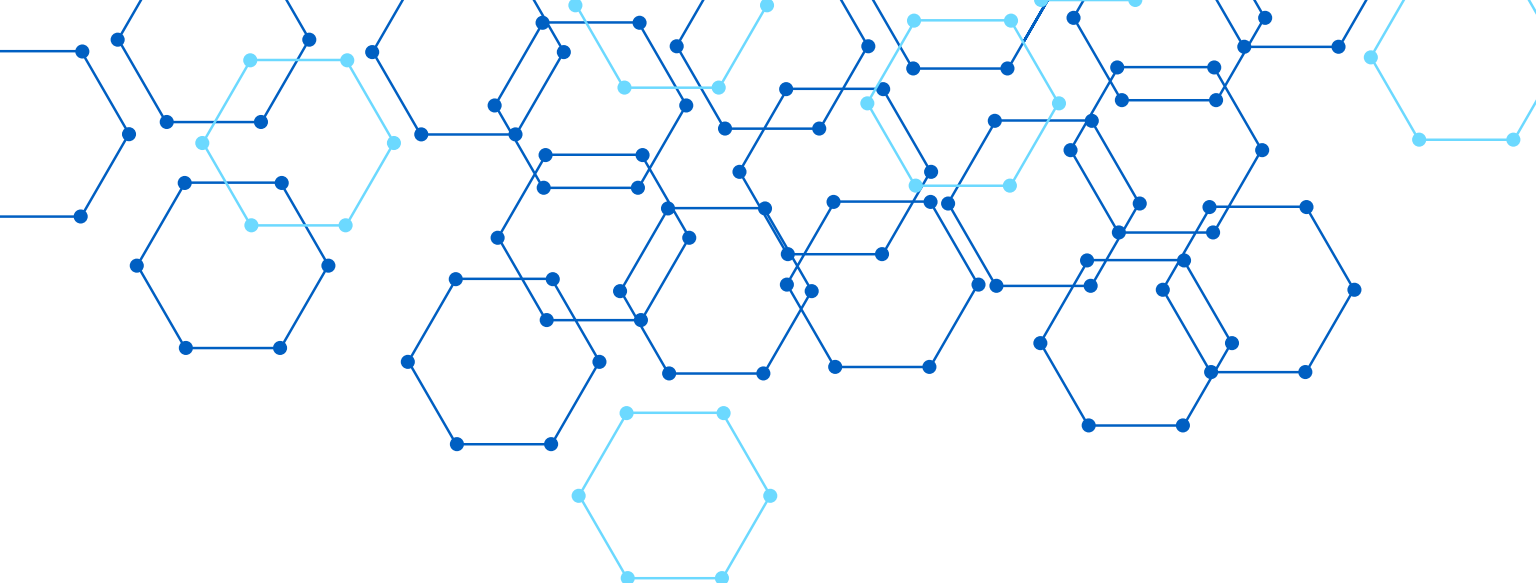
- Arquitectura modular en 4 capas bien definidas
 - Código abierto y fácilmente extensible
 - Interfaz intuitiva con Tkinter
 - Automatización de cálculos complejos
 - Detección y reporte de fallos de convergencia
- 





REFERENCIAS BIBLIOGRÁFICAS

- Chapra, S. C., & Canale, R. P. (2010). Métodos numéricos para ingenieros (6.^a ed.). McGraw-Hill. Disponible en: <https://pdfcoffee.com/metodos-numericos-para-ing-6a-chapra-9-pdf-free.html>
 - Muller, D. E. (1956). A method for solving algebraic equations using an automatic computer. Mathematics of Computation, 10(56), 208–215. Disponible en: <https://www.ams.org/journals/mcom/1956-10-056/S0025-5718-1956-0083822-0/S0025-5718-1956-0083822-0.pdf>
 - Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. Nature, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
 - Python Software Foundation. (2024). Python 3.11 documentation. <https://docs.python.org/3.11/>
- 
- 



MUCHAS GRACIAS

